PhyArc系列电机使用说明手册

1. 调试方法

1. 使用本公司所自研的调试控制板（需从本公司进行购买）搭配自研的调试SDK以及上位机调试软件进行调试。
2. 使用CANFD通信协议进行电机调试。

2. 注意事项：

1. 请按照本文规定的工作参数使用，否则可能会对本产品造成严重的损坏！
2. 在关节运行时不可切换控制方式，如需切换需要发送停止运行命令后再做切换。
3. 本产品使用应该远离强磁环境，否则会影响电机编码器数据，造成电机转动异常。
4. 本产品使用应保证其稳定在80°C以内，超过80°C电机会警报，超过电机驱动板温度高于105°C、电机线圈温度高于120°C电机会强行宕机，进入待机状态，无法进行控制，待温度正常后恢复正常。
5. 本产品在大扭矩工作状态时，会产生较多热量，注意避免被烫伤。
6. 本产品在调试或工作时，应保证外壳被固定。
7. 请勿自行拆卸本产品，否则会影响产品的正常使用，若私自拆卸则不再享受相应的质保服务。
8. 注意接线牢固，避免接触不良。

3. 安装说明

3.1 接线方式说明

PhyArc系列关节不同型号使用不同接口，接口设置可见产品数据手册(*可联系sales@phybot.cn，product@phybot.cn*)，主要分为三种：

1. PhyArc47使用本体引出两端线，一端为供电接口，一端为CANFD通讯接口。
2. PhyArc68,78具备两个XT30（2+2）接口，任意端可以作为电源和通信的输入或输出接口，另一端可以串联其他关节模组。
3. PhyArc95,102,具备两个XT30接口，任意端可以作为电源的输入或输出接口，另一端可以串联其他关节；具备3个Type-c作为通信接口，任意端可以作为通信的输入或输出接口，其余两端可以串联其他关节。

3.2 使用电压和电源说明

电机使用电压范围是72V，供电电压需要进行缓启动过程，逐渐升压至72V。在带载情况下，尽量使用电池供电，保证有足够的供电能力和能量回流渠道，若使用桌面实验室电源、开关电源、自制电源管理模块等设备供电时，注意加入足够耐压和容量的电容，保证能量能回流，否则易烧驱动，或者使得驱动报警或重启，重启是硬件保护的作用，若不重启，容易损坏驱动硬件导致无法再用。在使用桌面实验室电源、开关电源、自制电源管理模块等设备供电时，易导致驱动无法顺利开机启动，主要原因是驱动的电压保护机制，此类设备电压有爬坡过程，驱动判断为供电不稳定，解决方案为更换电池供电或联系官方技术支持。

3.3 通信使用说明

CANFD波特率为4M，标准帧，CANFD控制频率最高可达1000Hz。首次使用电机，建议查询电机ID或重置ID再进行控制，用户尽量在有经验情况下修改。配置好的电机等待驱动顺利开机后（至少10S）可直径发送控制指令，期间也可随时更改配置。对驱动发送如下设置指令时，需延迟500mS以上再发送第二个CAN指令。设置通信模式，设置当前点为零点，设置电机ID，若未延迟500ms以上，则可能设置失败。其他指令令则只需要不导致CAN繁忙即可。

4. 电机调试

4.1 上位机说明

4.1.1 版本说明：

版本	名称	发布日期	版本说明
V0.1.0	PhyArc调试软件	2025/9/7	支持PhyArc47，PhyArc68，PhyArc78，PhyArc95，PhyArc102多种型号，当前支持CANFD通信。

4.1.2 调试控制板连接

正确连接电机后，打开调试工具，进入到设置标定界面。首先进行主板对象初始化，需输入调试控制板ID：初始化完成后，可以进行主板IP及接口输入（为板端固定：），以及主机IP及端口输入（主机端口固定为：），点击连接即可连接至当前主板。若更换主板，需清楚当前主板对象后，重新进行操作。

4.1.3 获取和修改电机ID

完成主板连接后，可以点击获取，获取当前连接的电机ID，也可以通过输入主板接口ID和目标模组ID修改或设置当前的电机ID。注意主板ID请根据主板接口图进行确认和输入，电机ID范围为0~999，默认值为2，在进行此类操作时，请确保仅有一个电机连接至主板网络。

4.1.4 编码器标定

点击编码器标定按钮即启动编码器标定流程，通过观察编码器值变化，完成标定。

4.1.5 编码器标定

点击按钮即启动强拖，电机将进入开环转速转动模式，转速默认为2rad/s。可通过观察电机实际转动情况判断电机是否强拖成功。

4.1.6 设置零位

点击按钮即进行零位设置，将当前位置设置为零位。用户可以通过旁边的状态显示观察设置结果。

4.1.7 速度控制模式

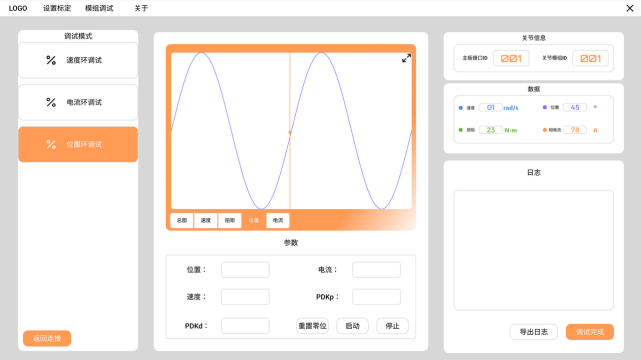
速度控制模式下用户可以通过调整速度，位置Kp（PosKp），速度Ki（VelKi）以及速度Kp（VelKp）这些参数来控制电机转动，界面上会显示电机转动实时的位置，速度，扭矩及电流。点击启动按钮，电机会开始转动，期间除观察数据外无法进行其他操作，点击停止则电机停止转动，其他操作解锁。可通过重置零位，将当前位置重新设为零点。

4.1.8 电流控制模式

电流控制模式下用户可以通过调整电流，位置Kp（PosKp），速度Ki（VelKi）以及速度Kp（VelKp）这些参数来控制电机转动，界面上会显示电机转动实时的位置，速度，扭矩及电流。点击启动按钮，电机会开始转动，期间除观察数据外无法进行其他操作，点击停止则电机停止转动，其他操作解锁。可通过重置零位，将当前位置重新设为零点。

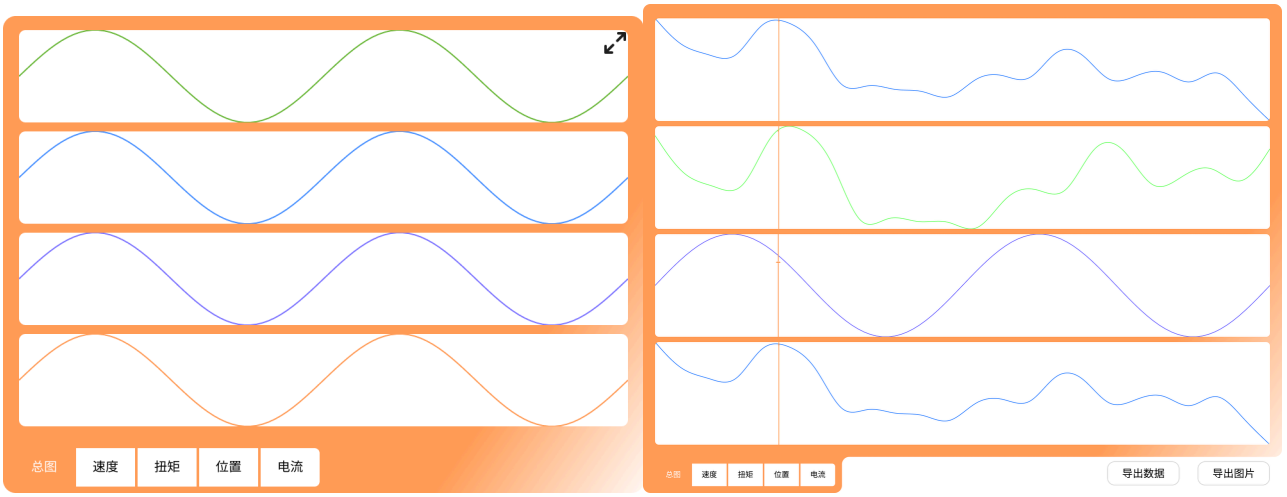
4.1.9 位置控制模式

位置控制模式下用户可以通过调整位置，速度，电流，PDKp以及PDKd这些参数来控制电机转动，界面上会显示电机转动实时的位置，速度，扭矩及电流。点击启动按钮，电机会开始转动，期间除观察数据外无法进行其他操作，点击停止则电机停止转动，其他操作解锁。可通过重置零位，将当前位置重新设为零点。



4.1.10 电机运行数据获取

电机数据支持大图查看，界面分为总图和速度，位置，电流，扭矩的分图，用户可以根据需要查看对应数据。



4.2 SDK说明：

4.2.1 版本说明：

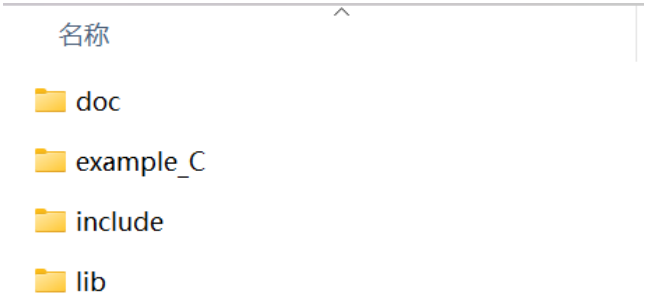
版本	名称	发布日期	版本说明
V0.1.0	MotorControl	2025/9/7	支持PhyArc47，PhyArc68，PhyArc78，PhyArc95，PhyArc102多种型号，当前支持CANFD通信。

4.2.2 SDK集成要求

系统	语言
Win10/Linux22.06.04	C/C++

4.2.3 SDK结构

sdk 目录包含了主控板 驱动程序的外部头文件 (inc 目录)，依赖库（lib目录），功能用例以及说明文档。



4.2.4 功能目录

功能	接口
初始化主板对象	RobotCtx * robot_create (unsigned short dev_id)
释放主板对象	void robot_destroy (RobotCtx *ctx)
连接主板	int robot_config_net (RobotCtx *ctx, const char *local_ip, int local_port, int remote_port, const char *remote_ip)
设置主板快速模式参数	int robot_set_fast_mode (RobotCtx *ctx, int enable, int period_ms)
创建单电机对象	RobotMotor * robot_create_motor (RobotCtx *ctx, unsigned short canId, unsigned short canLineId)

设置电机控制模式	int robot_motor_set_control_mode (RobotMotor *motor, unsigned int idx)
设置电机控制字	int robot_motor_set_control_world (RobotMotor *motor, unsigned int idx)
设置位置、速度和电流控制参数	void robot_motor_set_pos (RobotMotor *motor, float Pos, float Vel, float Cur)
根据电机列表组大包发送	void robot_motor_set_big_pose (RobotCtx *ctx)
获取PVCTFAST 位置、速度和电流控制参数	void robot_motor_get_PVCTFast (RobotMotor *motor, float *Pos, float *Vel, float *Cur)
获取PVCT 位置、速度和电流控制参数	void robot_motor_get_PVCT (RobotMotor *motor, float *Pos, float *Vel, float *Cur, float *Tor_l, float *Tor_e)
获取编码器值	void robot_motor_get_EncoderValue (RobotMotor *motor, int *EncoderValue)
设置强拖	int robot_motor_set_StrongDragging (RobotMotor *motor, unsigned int Anglesource, float Voltage)
获取电机强拖模式	int robot_motor_get_StrongDragging (RobotMotor *motor, unsigned int *Anglesource)
设置电机ID	int robot_motor_set_motor_id (RobotCtx *ctx, unsigned short Id)
获取电机ID	int robot_motor_get_motor_id (RobotCtx *ctx)
设置电机速度和电流控制参数	void robot_motor_set_vel (RobotMotor *motor, float Vel, float Cur)
设置电机电流控制参数	void robot_motor_set_cur (RobotMotor *motor, float Cur)
设置位置PD控制参数	void robot_motor_set_pd (RobotMotor *motor, float PDKp, float PDKd)
设置位置、速度PID控制参数	void robot_motor_set_pid (RobotMotor *motor, float PosKp, float VelKp, float VelKi)
释放单电机结构体	void robot_destroy_motor (RobotMotor *motor)

4.2.5 功能说明

功能接口	参数介绍
RobotCtx * robot_create (unsigned short dev_id)	Parameters dev_id 主板ID Returns 主板对象，失败返回nullptr
void robot_destroy (RobotCtx *ctx)	Parameters ctx 主板对象 Returns void
int robot_config_net (RobotCtx *ctx, const char *local_ip, int local_port, int remote_port, const char *remote_ip)	Parameters ctx 主板对象 local_ip 本地ip local_port 本地端口 remote_port 主板端口 remote_ip 主板ip Returns 连接是否成功，1 为成功，0为失败 ,-1 主板对象为nullptr
int robot_set_fast_mode (RobotCtx *ctx, int enable, int period_ms)	Parameters Pos 位置设定值 单位:rad Vel 速度设定值 单位:rad/s Cur 电流设定值 单位:A Returns void
RobotMotor * robot_create_motor (RobotCtx *ctx, unsigned short canId, unsigned short canLineId)	Parameters ctx 主板对象 canId canId motor 返回一个电机对象 canLineId can线Id Returns RobotMotor 类型指针

int robot_motor_set_control_mode (RobotMotor *motor, unsigned int idx)	Parameters motor 电机对象 idx 控制模式索引值 详见Types.h 文件 枚举 MotorCtrlMode_e Returns 1 设置成功 0 设置失败
int robot_motor_set_control_world (RobotMotor *motor, unsigned int idx)	Parameters motor 电机对象 idx 模式索引值 详见Types.h 文件 枚举 UserWorkMode_e Returns 1 设置成功 0 设置失败
void robot_motor_set_pos (RobotMotor *motor, float Pos, float Vel, float Cur)	Parameters Pos 位置设定值 单位:rad Vel 速度设定值 单位:rad/s Cur 电流设定值 单位:A Returns void
void robot_motor_set_big_pose (RobotCtx *ctx)	Parameters ctx 主板对象 Returns void
void robot_motor_get_PVCTFast (RobotMotor *motor, float *Pos, float *Vel, float *Cur)	Parameters Pos 位置设定值 单位:rad Vel 速度设定值 单位:rad/s Cur 电流设定值 单位:A Returns void
void robot_motor_get_PVCT (RobotMotor *motor, float *Pos, float *Vel, float *Cur, float *Tor_l, float *Tor_e)	Parameters Pos 存储位置值的引用 单位:rad Vel 存储速度值的引用 单位:rad/s Cur 存储电流值的引用 单位:A Tor_l 存储负载转矩值的引用 单位: N.m Tor_e 存储电磁转矩值的引用 单位: N.m

	Returns void
void robot_motor_get_EncoderValue (RobotMotor *motor, int *EncoderValue)	Parameters motor 电机对象 EncoderValue 编码器值 Returns void
int robot_motor_set_StrongDragging (RobotMotor *motor, unsigned int Anglesource, float Voltage)	Parameters motor 电机对象 Anglesource 设置强拖模式，见CTypes.h 枚举 ThetaSorce_e Voltage 电压 Returns
int robot_motor_get_StrongDragging (RobotMotor *motor, unsigned int *Anglesource)	Parameters motor 电机对象 Anglesource 设置强拖模式，见CTypes.h 枚举 ThetaSorce_e Returns 1 返回成功，0 返回失败
int robot_motor_set_motor_id (RobotCtx *ctx, unsigned short Id)	Parameters ctx 主板对象 Id 电机ID Returns 1 返回成功，0 返回失败， -1 主板对象为nullptr
int robot_motor_get_motor_id (RobotCtx *ctx)	Parameters ctx 主板对象 Returns 电机ID

void robot_motor_set_vel (RobotMotor *motor, float Vel, float Cur)	Parameters Vel 速度设定值 单位:rad/s Cur 电流设定值 单位:A Returns void
void robot_motor_set_cur (RobotMotor *motor, float Cur)	Parameters Cur 电流设定值 单位:A Returns void
void robot_motor_set_pd (RobotMotor *motor, float PDKp, float PDKd)	Parameters PDKp 位置环比例系数 PDKd 位置环微分系数 Returns void
void robot_motor_set_pid (RobotMotor *motor, float PosKp, float VelKp, float VelKi)	Parameters PosKp 位置环比例系数 VelKp 速度环比例系数 VelKi 速度环积分系数 Returns void
void robot_destroy_motor (RobotMotor *motor)	Parameters motor 电机指针 canId canId motor 返回一个电机对象 canLineId can线Id Returns void

详细数据可查看MotorControl: c/RobotControl.h File Reference

4.2.6 测试用例

SDK附带的测试用例均在SDK\example_C下，包含一下内容：

用例名	功能
current_control_get_pvct.c	电流控制测试用例
get_strong_dragging_model.c	强拖测试用例
pd_control_get_pvct.c	PD控制用例
set_control_world.c	设置控制符测试用例
set_id.c	设置ID测试用例
set_sensor_less.c	设置开环模式测试用例
velcity_control_get_pvct.c	速度控制测试用例

例：PD控制电机

```
/*
 * pd_control_get_pvct.c
 * 功能：对电机设置PD模式，使能电机，设置电机pd值，循环获取实际快速帧pvct
 * @author  xufuliang
 * @date    2025-08-27
 */
#include <stdio.h>
#include "RobotControl.h"
#include "CTypes.h"
#include <time.h>
#include <stdint.h>
#ifdef _WIN32
#include <windows.h>
```

```
#define SLEEP_POS 2000000   //2000*1000 为2000微妙=2ms 500hz
/* 返回自系统启动以来的纳秒数（单调递增） */
static inline uint64_t  nanos_now(void)
{
    static LARGE_INTEGER freq = { 0 };
    if (freq.QuadPart == 0)          /* 只需第一次调用时初始化 */
        QueryPerformanceFrequency(&freq);

    LARGE_INTEGER cnt;
    QueryPerformanceCounter(&cnt);

    /* 计数器差值 ÷ 频率 × 1e9  => 纳秒 */
    return (uint64_t)(cnt.QuadPart * 1000000000ULL / freq.QuadPart);
}

/* 睡眠指定的纳秒数（busy-wait 最后几百微秒） */
void sleep_ns(unsigned ns) {
    const uint64_t t0 = nanos_now();
    const uint64_t t_end = t0 + ns;
    /* 1. 如果剩余时间 > 1 ms，用 Sleep 先“粗睡” */
    if (ns >= 1 * 1000000ULL) {
        DWORD ms = (DWORD)((t_end - nanos_now()) / 1000000ULL);
        if (ms) Sleep(ms - 1);      /* 留 1 ms 给下面的忙等 */
    }
    /* 2. 忙等直到准确时刻 */
    while (nanos_now() < t_end)
        YieldProcessor();           /* VS 也可 _mm_pause(); */
}
void format_ns(uint64_t ns, char* buf, size_t len)
{
```

```
    uint64_t h = ns / 3600000000000ULL;
    ns %= 3600000000000ULL;
    uint64_t m = ns / 60000000000ULL;
    ns %= 60000000000ULL;
    uint64_t s = ns / 1000000000ULL;
    ns %= 1000000000ULL;
    snprintf(buf, len,
        "%02llu:%02llu:%02llu.%09llu",
        h, m, s, ns);
}
#else
#include <sys/time.h>
#define SLEEP_POS 2000000   //2000*1000 Ï2000[?]  =2ms 500hz
static inline uint64_t nanos_now(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC_RAW, &ts);
    return (uint64_t)ts.tv_sec * 1000000000ULL + ts.tv_nsec;
}

static inline void sleep_ns(uint64_t ns)
{
    struct timespec req = { ns / 1000000000ULL, ns % 1000000000ULL };
    nanosleep(&req, NULL);
}

static  uint64_t ts2ns(const struct timespec* ts)
{

    return (uint64_t)ts->tv_sec*1000000000ULL + ts->tv_nsec;
```

```
}

#endif

int main(int argc, char* argv[]) {

    //生成主板对象
    RobotCtx* ctx = robot_create(0xFD);
    if (!ctx) return -1;
    //连接主板
    int value = robot_config_net(ctx, "192.168.3.245", 15021, 14999, "192.168.3.11");
    if (value)
        {
                printf("连接主板成功\n");
        }
        else
        {
                printf("连接主板失败\n");
        }
    //生成电机对象
    RobotMotor* motor = robot_create_motor(ctx, 12, 1);



    //设置主板上传频率 为500hz
    value = robot_set_fast_mode(ctx, 1, 500);
    if (value)
    {
        printf("设置主板上传频率成功\n");
    }
    else
```

```
{
    printf("设置主板上传频率失败\n");
}
//设置电机PD模式
value = robot_motor_set_control_mode(motor, MOTOR_CTRL_MODE_PD);
if(value)
{
    printf("设置电机pd模式成功\n");
}
else
{
    printf("设置电机pd模式失败\n");
}
//为更新电机参数设置pos
robot_motor_set_pos(motor, 0, 0, 0);
for (int i = 0; i < 2; i++)
{
#ifdef _WIN32
    char str[32];
    format_ns(nanos_now(), str, sizeof(str));
    puts(str);
    uint64_t t0 = nanos_now();
    robot_motor_set_big_pose(ctx);
    uint64_t t1 = nanos_now();
    //printf("CPU: %llu ns\n", (uint64_t)(t1 - t0));
    //printf("shijiancha:%llu ns\n", SLEEP_POS - ((uint64_t)(t1 - t0)));
    sleep_ns(SLEEP_POS - ((uint64_t)(t1 - t0)));

#else
    struct timespec t1;
```

```
    struct timespec t2;
    clock_gettime(CLOCK_REALTIME, &t1);
    robot_motor_set_big_pose(ctx);
    clock_gettime(CLOCK_REALTIME, &t2);
    double total = (double)(ts2ns(&t2) - ts2ns(&t1)) / CLOCKS_PER_SEC;
    printf("CPU:%g\n", total);

    sleep_ns((SLEEP_POS-total));
#endif
}

//设置电机使能
value = robot_motor_set_control_world(motor, CTRL_SERVO_ON);
if (value)
{
    printf("设置电机使能 成功\n");
}
else
{
    printf("设置电机使能 失败\n");
}

robot_motor_set_pd(motor,50,5);
float pos=0.0, vel=0.0, cur=0.0;
//获取电机快速模式的pvct值
robot_motor_get_PVCTFast(motor, &pos, &vel, &cur);
printf("pos:%f,vel:%f,tur:%f\n", pos, vel, cur);
//释放电机对象
robot_destroy_motor(motor);
//释放主板对象
```

```
    robot_destroy(ctx);
    return 0;
}
```

5. CANFD通信使用协议说明

5.1 CANFD通信参数

波特率： 仲裁段和ACK段1Mbit/s 数据段5Mbit/s。

通信模式： 问答模式。

电机默认ID: 0x02

电机广播ID: 0xFE

5.2 包格式：

CANID 1	命令2	读/写1	数据
--	CH,CL	--	N

目标设备: 1个字节，电机板 ID

命令：2个字节

读/写：03/06

数据： N

5.3 电机设置指令：

5.3.1 电机零点设置指令

命令：0x603

主控发送：

CANID 1	命令2	读/写1	数据
--	0x603	0x06	0x102F

电机反馈：

帧头2	数据长度2	CANID 2	命令2	读/写1	状态	CRC16校验2
AA F3	0x00 0x06	--	0x602	0x06	0x01	--

电机反馈为确认成功ACK

反馈数据长度为：从CANID到CRC16校验前

5.3.2 电机CAN通信ID设置指令

命令：0x601

主控发送：

CANID 1	命令2	读/写1	数据 2
0xFE/旧ID	0x601	0x06	新CANID2字节

电机反馈：

帧头2	数据长度2	CANID 2	命令2	读/写1	状态	CRC16校验2
AA F3	0x00 0x06	--	0x601	0x06	0x01	--

电机反馈为确认成功ACK

主控发送数据部分CANID为俩个字节，低字节有效

5.3.3 电机CAN通信ID获取指令

命令：0x601

主控发送：

CANID 1	命令2	读/写1
0xFE/旧ID	0x601	0x03

电机反馈：

帧头2	数据长度2	CANID 2	命令2	读/写1	数据2	CRC16校验2
AA F3	0x00 0x0D	0x00 0xFE	0x601	0x03	CANID 2字节	--

电机反馈： 数据部分为当前电机ID 0xFE为广播ID

5.4 电机控制指令：

5.4.1 电机使能：

命令：0x603

主控发送：

CANID 1	命令2	读/写1	数据
--	0x603	0x06	0xF002

电机反馈：

帧头2	数据长度2	CANID 2	命令2	读/写1	状态	CRC16校验2
AA F3	0x00 0x06	--	0x603	0x06	0x01	--

电机反馈为确认成功ACK

反馈数据长度为：从CANID到CRC16校验前

5.4.2 电机使能：

命令：0x603

主控发送：

CANID 1	命令2	读/写1	数据
--	0x603	0x06	0xF002

电机反馈：

帧头2	数据长度2	CANID 2	命令2	读/写1	状态	CRC16校验2
AA F3	0x00 0x06	--	0x603	0x06	0x01	--

电机反馈为确认成功ACK

反馈数据长度为：从CANID到CRC16校验前

5.4.3 设置电机运行模式：

命令：0x602

命令	含义	读/写　0x03/0x06	说明
0x300	帧ACK应答		确认接收
0x301	修改主板设备ID	0x06	修改主板设备ID
0x302	修改从设备ID	0x06	修改从设备ID
0x303	修改主板IP和端口	0x06	修改主板IP和端口
0x5FF	修改所有从设备配置	0x06	修改所有从设备配置

主控发送：

CANID 1	命令2	读/写1	数据
--	0x603	0x06	--

电机反馈：

帧头2	数据长度2	CANID 2	命令2	读/写1	状态	CRC16校验2
AA F3	0x00 0x06	--	0x603	0x06	0x01	--

电机反馈为确认成功ACK

反馈数据长度为：从CANID到CRC16校验前

命令	含义	读/写　0x03/0x06	说明
0x300	帧ACK应答		确认接收
0x301	修改主板设备ID	0x06	修改主板设备ID
0x302	修改从设备ID	0x06	修改从设备ID
0x303	修改主板IP和端口	0x06	修改主板IP和端口
0x5FF	修改所有从设备配置	0x06	修改所有从设备配置

0x300 帧ACK应答： 状态： 0x01 确认接收

帧头2	数据长度2	源设备1	目标设备1	命令2	读/写1	状态	CRC16校验2
AA F2	0x0C	--	--	0x100	0x03	--	--

0x5FF 广播到所有从设备

帧头2	数据长度2	源设备1	目标设备1	命令2	读/写1	状态	CRC16校验2
AA F2	--	--	--	**0x5FF**	0x06	--	--

5.5 命令基地址：

命令	含义	读/写　0x03/0x06	说明
0x600	ActuatorBaseParams	0x03/0x06	执行器基础参数
0x601	ActuatorSensorParams	0x03/0x06	执行器传感器参数
0x602	ActuatorControlParams	0x03/0x06	执行器控制参数
0x603	ActuatorFuncParams	0x03/0x06	执行器功能参数

0x604	MotorParams	0x03/0x06	电机参数
0x605	HardwareParams	0x03/0x06	硬件参数
0x606	ForceParams	0x03/0x06	强拖控制参数
0x607	FulxObsParams	0x03/0x06	无感控制参数
0x608	FrictionParams	0x03	摩擦补偿参数
0x609	ErrDectParams	0x03	错误检测参数
0x60A	TempObsParams	0x03/0x06	绕组温度电流参数
0x60B	TorqueCalibParams	0x03	电流力矩标定参数
0x60C	CoilNTCParams	0x03/0x06	绕组NTC参数
0x60D	MosNTCParams	0x03/0x06	MOS关NTC参数
0x60E	MotorAngleSensorParams	0x03	电机角度传感器参数
0x60F	OutPutAngleSensorParams	0x03	输出角度传感器参数
0x610	CtrlConfig	0x03/0x06	实时控制配置
0x611	SetPVCT	0x06	PVCT指令
0x612	GetPVCT	0x03	PVCT反馈
0x613	SetPIDParam	0x06	PID参数指令
0x614	GetErrCode	0x03	错误码反馈
0x615	GetInverterStatus	0x03	逆变器状态反馈